

# Seance 4

Smart Pointers et Gestion Memoire

Ne plus jamais oublier de delete

Cours de renforcement C++

## Table des matières

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction : Le probleme avec les pointeurs classiques</b>      | <b>3</b>  |
| 1.1      | Rappel des seances precedentes . . . . .                             | 3         |
| 1.2      | Exemple de catastrophe . . . . .                                     | 3         |
| 1.3      | La solution moderne : Les Smart Pointers . . . . .                   | 4         |
| <b>2</b> | <b>unique_ptr : La propriete exclusive</b>                           | <b>5</b>  |
| 2.1      | Le concept . . . . .                                                 | 5         |
| 2.2      | Creation d'un unique_ptr . . . . .                                   | 5         |
| 2.3      | Utilisation de unique_ptr . . . . .                                  | 5         |
| 2.4      | Impossible de copier un unique_ptr . . . . .                         | 6         |
| 2.5      | Transférer la propriete avec std::move . . . . .                     | 6         |
| 2.6      | Retourner un unique_ptr . . . . .                                    | 7         |
| 2.7      | unique_ptr avec des tableaux . . . . .                               | 7         |
| <b>3</b> | <b>shared_ptr : La propriete partagee</b>                            | <b>9</b>  |
| 3.1      | Le concept . . . . .                                                 | 9         |
| 3.2      | Comment ca marche en coulisse? . . . . .                             | 9         |
| 3.3      | Creation et utilisation . . . . .                                    | 9         |
| 3.4      | Quand utiliser shared_ptr? . . . . .                                 | 10        |
| 3.5      | Le cout de shared_ptr . . . . .                                      | 11        |
| <b>4</b> | <b>weak_ptr : Observer sans posseder</b>                             | <b>12</b> |
| 4.1      | Le probleme des references circulaires . . . . .                     | 12        |
| 4.2      | La solution : weak_ptr . . . . .                                     | 12        |
| 4.3      | Utiliser un weak_ptr . . . . .                                       | 13        |
| 4.4      | Cas d'usage : Cache avec eviction . . . . .                          | 14        |
| <b>5</b> | <b>Tableau recapitulatif : Quel smart pointer utiliser ?</b>         | <b>15</b> |
| <b>6</b> | <b>Exemples pratiques complets</b>                                   | <b>16</b> |
| 6.1      | Exemple 1 : Gestionnaire de plugins avec unique_ptr . . . . .        | 16        |
| 6.2      | Exemple 2 : Graphe avec shared_ptr et weak_ptr . . . . .             | 17        |
| 6.3      | Exemple 3 : Factory retournant unique_ptr . . . . .                  | 18        |
| <b>7</b> | <b>Exercices pratiques</b>                                           | <b>20</b> |
| 7.1      | Exercice 1 : Convertir raw pointers en unique_ptr (facile) . . . . . | 20        |
| 7.2      | Exercice 2 : Gestionnaire de ressources (moyen) . . . . .            | 20        |
| 7.3      | Exercice 3 : Arbre avec shared_ptr (difficile) . . . . .             | 22        |

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| <b>8</b>  | <b>Pieges et bonnes pratiques</b>                                 | <b>24</b> |
| 8.1       | Piege 1 : Creer deux shared_ptr independants . . . . .            | 24        |
| 8.2       | Piege 2 : Retourner this dans shared_ptr . . . . .                | 24        |
| 8.3       | Piege 3 : Stocker shared_ptr dans un weak_ptr trop tard . . . . . | 24        |
| 8.4       | Bonne pratique 1 : Preferer make_unique et make_shared . . . . .  | 25        |
| 8.5       | Bonne pratique 2 : Passer les smart pointers . . . . .            | 25        |
| <b>9</b>  | <b>Quiz de validation</b>                                         | <b>26</b> |
| 9.1       | Questions . . . . .                                               | 26        |
| 9.2       | Reponses . . . . .                                                | 26        |
| <b>10</b> | <b>Pour aller plus loin</b>                                       | <b>27</b> |
| 10.1      | Custom deleters . . . . .                                         | 27        |
| 10.2      | enable_shared_from_this . . . . .                                 | 27        |
| 10.3      | Atomic shared_ptr (C++20) . . . . .                               | 27        |

# 1 Introduction : Le probleme avec les pointeurs classiques

## 1.1 Rappel des seances precedentes

Dans les Seances 1 a 3, nous avons beaucoup utilise des pointeurs, notamment avec les interfaces :

```
1 class ILogger {
2 public:
3     virtual void log(const std::string& msg) = 0;
4     virtual ~ILogger() = default;
5 };
6
7 class FileLogger : public ILogger {
8 private:
9     std::ofstream file;
10 public:
11     FileLogger(const std::string& filename) {
12         file.open(filename);
13     }
14     void log(const std::string& msg) override {
15         file << msg << "\n";
16     }
17 };
18
19 void utiliserLogger() {
20     ILogger* logger = new FileLogger("app.log");
21     logger->log("Message");
22     delete logger; // ATTENTION : N'oubliez pas cette ligne !
23 }
```

### Les problemes avec new/delete

1. **Oubli de delete** : Fuite memoire garantie
2. **Exception entre new et delete** : La memoire n'est jamais liberee
3. **Double delete** : Crash du programme
4. **Delete puis utilisation** : Comportement indefini (crash probable)
5. **Gestion manuelle complexe** : Qui est responsable du delete ?

## 1.2 Exemple de catastrophe

```
1 void fonctionDangereuse() {
2     ILogger* logger = new FileLogger("app.log");
3
4     logger->log("Debut du traitement");
5
6     // Beaucoup de code...
7
8     if (erreur) {
9         return; // OUPS ! On a oublie de delete logger !
10    }
11
12    // Encore du code...
13
14    if (autreErreur) {
```

```

15         throw std::runtime_error("Erreur");
16         // OUPS ! Exception, on saute le delete !
17     }
18
19     delete logger; // On n'arrive jamais ici si erreur...
20 }
21
22 // Resultat : FUIITE MEMOIRE a chaque appel avec erreur !

```

#### La question fondamentale

**Comment garantir qu'un objet alloue dynamiquement sera TOUJOURS libere, meme en cas d'exception ou de return premature ?**

**Reponse : RAII + Smart Pointers !**

### 1.3 La solution moderne : Les Smart Pointers

Un smart pointer est un objet qui se comporte comme un pointeur, mais qui libere automatiquement la memoire dans son destructeur.

```

1 void fonctionSecurisee() {
2     std::unique_ptr<ILogger> logger =
3         std::make_unique<FileLogger>("app.log");
4
5     logger->log("Debut du traitement");
6
7     if (erreur) {
8         return; // OK ! Le destructeur de unique_ptr delete
9                 // automatiquement
10    }
11
12    if (autreErreur) {
13        throw std::runtime_error("Erreur");
14        // OK ! Le destructeur est appele pendant le stack unwinding
15    }
16
17    // Pas besoin de delete !
18 } // <- Logger libere automatiquement ici

```

#### Les avantages

- **Impossible d'oublier le delete** : C'est automatique
- **Exception-safe** : La memoire est liberee meme en cas d'exception
- **Clarte** : Le code montre clairement qui possede l'objet
- **Prevention des bugs** : Pas de double delete, pas de dangling pointers

## 2 unique\_ptr : La propriete exclusive

### 2.1 Le concept

`unique_ptr` represente une propriete **exclusive** d'un objet.

**En langage simple** : "Cet objet m'appartient a MOI et a MOI SEUL. Quand je meurs, l'objet meurt avec moi."

```
1 #include <memory>
2
3 std::unique_ptr<int> ptr = std::make_unique<int>(42);
4 // ptr possede EXCLUSIVEMENT cet entier
5
6 std::cout << *ptr << "\n"; // 42
7
8 // Quand ptr sort de la portee, l'entier est automatiquement libere
```

#### Proprietes de `unique_ptr`

- **Un seul proprietaire** : On ne peut pas copier un `unique_ptr`
- **Liberation automatique** : Le `delete` est appele dans le destructeur
- **Zero overhead** : Aussi rapide qu'un pointeur brut
- **Transferable** : On peut transferer la propriete avec `std::move`

### 2.2 Creation d'un `unique_ptr`

Methode moderne (recommandee) :

```
1 auto ptr = std::make_unique<int>(42);
2 auto logger = std::make_unique<FileLogger>("app.log");
3 auto vec = std::make_unique<std::vector<int>>(10, 0);
```

Methode ancienne (C++11, avant `make_unique`) :

```
1 std::unique_ptr<int> ptr(new int(42));
2 // Moins sure, evitez si possible
```

#### Pourquoi `make_unique` ?

**Plus sur** : Evite les fuites memoire en cas d'exception

**Plus lisible** : Pas besoin de repeter le type

**Exception-safe** : Garantit que la memoire est liberee meme si le constructeur echoue

### 2.3 Utilisation de `unique_ptr`

```
1 class Personne {
2 private:
3     std::string nom;
4 public:
5     Personne(const std::string& n) : nom(n) {
6         std::cout << nom << " cree\n";
7     }
8     ~Personne() {
9         std::cout << nom << " detruit\n";
10    }
```

```

11     void saluer() const {
12         std::cout << "Bonjour, je suis " << nom << "\n";
13     }
14 };
15
16 int main() {
17     std::unique_ptr<Personne> alice =
18         std::make_unique<Personne>("Alice");
19
20     // Utilisation comme un pointeur normal
21     alice->saluer();           // Via ->
22     (*alice).saluer();        // Via *
23
24     // Verifier si le pointeur est non-null
25     if (alice) {
26         std::cout << "alice existe\n";
27     }
28
29     // Obtenir le pointeur brut (attention !)
30     Personne* raw = alice.get();
31
32 } // Alice est automatiquement detruite ici
33
34 // Affiche :
35 // Alice cree
36 // Bonjour, je suis Alice
37 // alice existe
38 // Alice detruit

```

## 2.4 Impossible de copier un unique\_ptr

```

1 std::unique_ptr<int> ptr1 = std::make_unique<int>(42);
2
3 // std::unique_ptr<int> ptr2 = ptr1; // ERREUR de compilation !
4 // "unique_ptr cannot be copied"
5
6 // Pourquoi ? Car sinon, qui serait responsable du delete ?
7 // Les deux ? Un seul ? -> Ambiguite !

```

### Le principe UNIQUE

**unique = un seul propriétaire**

Si on pouvait copier, il y aurait deux propriétaires, et ce ne serait plus “unique”!

C'est une protection du compilateur contre les bugs.

## 2.5 Transférer la propriété avec std::move

Si on ne peut pas copier, comment passer un unique\_ptr à une fonction ?

**Solution : Transférer la propriété avec std::move**

```

1 void prendrePossession(std::unique_ptr<int> ptr) {
2     std::cout << "J'ai reçu : " << *ptr << "\n";
3     // ptr possède maintenant l'objet
4 } // ptr est détruit ici, donc l'objet aussi
5
6 int main() {

```

```

7   std::unique_ptr<int> ptr = std::make_unique<int>(42);
8
9   std::cout << "Avant move\n";
10
11  prendrePossession(std::move(ptr));
12  // std::move TRANSFERE la propriete
13  // ptr ne possede plus rien maintenant !
14
15  std::cout << "Apres move\n";
16
17  if (!ptr) {
18      std::cout << "ptr est vide maintenant\n";
19  }
20 }
21
22 // Affiche :
23 // Avant move
24 // J'ai recu : 42
25 // Apres move
26 // ptr est vide maintenant

```

#### std::move en detail

**std::move(ptr)** dit : “Je transfere la propriete de cet objet. Je n’en veux plus.”

Apres le move :

- L’ancien proprietaire (ptr) devient vide (nullptr)
- Le nouveau proprietaire possede l’objet
- Il n’y a toujours qu’UN SEUL proprietaire a la fois

**Metaphore :** C’est comme donner les cle de votre voiture a quelqu’un. Vous n’avez plus les cle, donc vous ne pouvez plus utiliser la voiture.

## 2.6 Retourner un unique\_ptr

```

1  std::unique_ptr<Personne> creerPersonne(const std::string& nom) {
2      auto p = std::make_unique<Personne>(nom);
3      // Faire des trucs avec p...
4      return p; // OK ! Le compilateur fait le move automatiquement
5  }
6
7  int main() {
8      auto alice = creerPersonne("Alice");
9      alice->saluer();
10 }

```

#### Return Value Optimization

Quand vous retournez un unique\_ptr, le compilateur fait automatiquement un “move”. Vous n’avez pas besoin d’ecrire `return std::move(p)`.

En fait, ecrire `std::move` dans un return peut meme EMPECHER certaines optimisations !

**Regle :** Retournez simplement la variable, le compilateur s’occupe du reste.

## 2.7 unique\_ptr avec des tableaux

```

1 // Ancien style (C)
2 int* tableau = new int[100];
3 // ... utilisation ...
4 delete[] tableau; // N'oubliez pas le [] !
5
6 // Nouveau style (C++)
7 auto tableau = std::make_unique<int[]>(100);
8 // ... utilisation ...
9 // Liberation automatique avec delete[]
10
11 tableau[0] = 42;
12 tableau[99] = 10;
13
14 // Pas besoin de delete[] : automatique !

```

Mais en vrai...

**Pour les tableaux, utilisez plutôt std::vector !**

```

1 std::vector<int> tableau(100);
2 tableau[0] = 42;
3 // Beaucoup plus flexible et sur

```

unique\_ptr avec tableau est utile seulement dans des cas très spécifiques (interfacage avec du code C, par exemple).

## 3 shared\_ptr : La propriete partagee

### 3.1 Le concept

**shared\_ptr** represente une propriete **partagee** d'un objet.

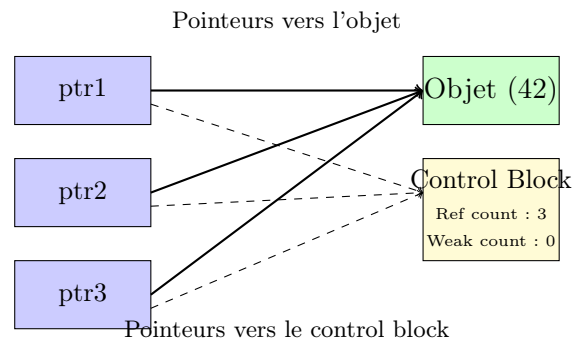
**En langage simple :** "Plusieurs personnes peuvent posseder cet objet. Il ne sera detruit que quand le DERNIER proprietaire mourra."

```
1 #include <memory>
2
3 std::shared_ptr<int> ptr1 = std::make_shared<int>(42);
4 std::shared_ptr<int> ptr2 = ptr1; // Copie OK !
5 std::shared_ptr<int> ptr3 = ptr1; // Encore une copie !
6
7 // Maintenant 3 shared_ptr pointent vers le meme int
8 // L'entier sera detruit quand les 3 seront detruits
9
10 std::cout << ptr1.use_count() << "\n"; // 3
```

#### Proprietes de shared\_ptr

- **Plusieurs proprietaires** : On peut copier un shared\_ptr
- **Reference counting** : Le shared\_ptr compte combien de copies existent
- **Liberation automatique** : L'objet est detruit quand le compteur atteint 0
- **Thread-safe** : Le compteur est protege pour les acces concurrents

### 3.2 Comment ca marche en coulisse ?



Chaque `shared_ptr` contient :

1. Un pointeur vers l'objet
2. Un pointeur vers un "control block" qui contient le compteur de references

Quand on copie un `shared_ptr`, le compteur augmente. Quand un `shared_ptr` est detruit, le compteur diminue. Quand le compteur atteint 0, l'objet est detruit.

### 3.3 Creation et utilisation

```
1 class Fichier {
2 private:
3     std::string nom;
4 public:
5     Fichier(const std::string& n) : nom(n) {
6         std::cout << "Ouverture de " << nom << "\n";
```

```

7     }
8     ~Fichier() {
9         std::cout << "Fermeture de " << nom << "\n";
10    }
11    void lire() {
12        std::cout << "Lecture de " << nom << "\n";
13    }
14 };
15
16 int main() {
17     std::shared_ptr<Fichier> ptr1 =
18         std::make_shared<Fichier>("data.txt");
19
20     std::cout << "Compteur : " << ptr1.use_count() << "\n"; // 1
21
22     {
23         std::shared_ptr<Fichier> ptr2 = ptr1; // Copie
24         std::cout << "Compteur : " << ptr1.use_count() << "\n"; // 2
25
26         ptr2->lire();
27
28     } // ptr2 detruit ici, compteur diminue
29
30     std::cout << "Compteur : " << ptr1.use_count() << "\n"; // 1
31
32 } // ptr1 detruit ici, compteur atteint 0, fichier ferme
33
34 // Affiche :
35 // Ouverture de data.txt
36 // Compteur : 1
37 // Compteur : 2
38 // Lecture de data.txt
39 // Compteur : 1
40 // Fermeture de data.txt

```

### 3.4 Quand utiliser shared\_ptr ?

Cas d'usage typiques :

#### 1. Graphes et structures partagees

```

1 class Noeud {
2 public:
3     std::vector<std::shared_ptr<Noeud>> voisins;
4     std::string nom;
5
6     Noeud(const std::string& n) : nom(n) {}
7 };
8
9 auto a = std::make_shared<Noeud>("A");
10 auto b = std::make_shared<Noeud>("B");
11 auto c = std::make_shared<Noeud>("C");
12
13 // A pointe vers B et C
14 a->voisins.push_back(b);
15 a->voisins.push_back(c);
16
17 // B pointe vers C
18 b->voisins.push_back(c);

```

```

19
20 // C est possede par A et B
21 // Il ne sera detruit que quand A ET B seront detruits

```

## 2. Cache partage

```

1 class Cache {
2 private:
3     std::map<std::string, std::shared_ptr<Data>> data;
4
5 public:
6     std::shared_ptr<Data> get(const std::string& key) {
7         if (data.find(key) != data.end()) {
8             return data[key]; // Copie du shared_ptr
9         }
10        // Charger depuis le disque...
11        auto d = std::make_shared<Data>(/* ... */);
12        data[key] = d;
13        return d;
14    }
15 };
16
17 // Plusieurs utilisateurs peuvent avoir le meme Data
18 // Il reste en memoire tant que quelqu'un l'utilise

```

## 3. Callbacks et closures

```

1 class Button {
2 private:
3     std::function<void()> onClick;
4     std::shared_ptr<Context> context;
5
6 public:
7     void setOnClick(std::shared_ptr<Context> ctx,
8                     std::function<void()> callback) {
9         context = ctx;
10        onClick = callback;
11    }
12
13    void click() {
14        if (onClick) onClick();
15    }
16 };

```

### 3.5 Le cout de shared\_ptr

#### Overhead de shared\_ptr

**Memoire** : 2 pointeurs au lieu d'1 (objet + control block)

**Performance** : Incrementation/decrementation atomique du compteur (thread-safe = leger cout)

**Allocation** : Allocation du control block (une fois)

**Conclusion** : Utilisez shared\_ptr quand vous en avez VRAIMENT besoin (propiete partagee). Sinon, unique\_ptr est plus leger.

## 4 weak\_ptr : Observer sans posséder

### 4.1 Le probleme des references circulaires

```
1 class B; // Declaration anticipee
2
3 class A {
4 public:
5     std::shared_ptr<B> ptrB;
6     ~A() { std::cout << "A detruit\n"; }
7 };
8
9 class B {
10 public:
11     std::shared_ptr<A> ptrA; // Reference circulaire !
12     ~B() { std::cout << "B detruit\n"; }
13 };
14
15 int main() {
16     auto a = std::make_shared<A>();
17     auto b = std::make_shared<B>();
18
19     a->ptrB = b; // A possede B (ref count de B = 2)
20     b->ptrA = a; // B possede A (ref count de A = 2)
21
22 } // FUIITE MEMOIRE ! Ni A ni B ne sont detruits !
23
24 // A ne peut pas etre detruit car B le possede
25 // B ne peut pas etre detruit car A le possede
26 // -> Reference circulaire -> FUIITE
```

#### Le piege

Deux `shared_ptr` qui se pointent mutuellement = fuite memoire garantie !  
Le compteur de references ne descendra jamais a 0.

### 4.2 La solution : weak\_ptr

`weak_ptr` est un pointeur qui "observe" un objet sans le posseder.

En langage simple : "Je veux pouvoir acceder a cet objet, mais je ne veux pas empecher sa destruction."

```
1 class A {
2 public:
3     std::shared_ptr<B> ptrB;
4     ~A() { std::cout << "A detruit\n"; }
5 };
6
7 class B {
8 public:
9     std::weak_ptr<A> ptrA; // Observer, pas posseder !
10    ~B() { std::cout << "B detruit\n"; }
11 };
12
13 int main() {
14     auto a = std::make_shared<A>();
15     auto b = std::make_shared<B>();
```

```

16     a->ptrB = b;           // A possede B (ref count B = 2)
17     b->ptrA = a;           // B observe A (ref count A = 1 !)
18
19
20 } // A detruit (ref count = 0), puis B detruit
21
22 // Affiche :
23 // A detruit
24 // B detruit

```

### 4.3 Utiliser un weak\_ptr

**Probleme :** Un weak\_ptr ne peut pas etre dereference directement !

Pourquoi ? Car l'objet pointe peut avoir ete detruit entre-temps.

```

1  std::weak_ptr<int> weak;
2
3  {
4      auto shared = std::make_shared<int>(42);
5      weak = shared; // weak observe shared
6
7      std::cout << "Dans le bloc\n";
8      // On ne peut PAS faire : *weak
9      // weak n'est qu'un observateur !
10
11 } // shared detruit ici, l'objet n'existe plus
12
13 std::cout << "Hors du bloc\n";
14 // weak pointe maintenant vers un objet detruit !
15 // Comment savoir si l'objet existe encore ?

```

**Solution :** Convertir weak\_ptr en shared\_ptr

```

1  std::weak_ptr<int> weak;
2
3  {
4      auto shared = std::make_shared<int>(42);
5      weak = shared;
6
7      // Pour utiliser weak, on le convertit en shared_ptr
8      if (auto locked = weak.lock()) {
9          // locked est un shared_ptr temporaire
10         std::cout << *locked << "\n"; // 42
11         // L'objet est garanti d'exister pendant ce bloc
12     } else {
13         std::cout << "L'objet n'existe plus\n";
14     }
15 }
16
17 // Hors du bloc
18 if (auto locked = weak.lock()) {
19     std::cout << *locked << "\n";
20 } else {
21     std::cout << "L'objet n'existe plus\n"; // <- Affiche ca
22 }

```

`weak.lock()`

**`weak.lock()`** essaie de creer un `shared_ptr` temporaire :

- Si l'objet existe encore : retourne un `shared_ptr` valide
- Si l'objet est detruit : retourne un `shared_ptr` vide (`nullptr`)

**Pattern idiomatique :**

```
1 if (auto ptr = weak.lock()) {  
2     // Utiliser ptr en toute securite  
3 } else {  
4     // L'objet n'existe plus  
5 }
```

#### 4.4 Cas d'usage : Cache avec eviction

```
1 class ImageCache {  
2 private:  
3     std::map<std::string, std::weak_ptr<Image>> cache;  
4  
5 public:  
6     std::shared_ptr<Image> get(const std::string& filename) {  
7         // Verifier si l'image est en cache  
8         if (cache.find(filename) != cache.end()) {  
9             if (auto img = cache[filename].lock()) {  
10                 std::cout << "Image trouvee en cache\n";  
11                 return img; // Image encore en memoire  
12             }  
13         }  
14  
15         // Image pas en cache ou expiree, charger depuis le disque  
16         std::cout << "Chargement depuis le disque\n";  
17         auto img = std::make_shared<Image>(filename);  
18         cache[filename] = img; // Stocker weak_ptr  
19         return img;  
20     }  
21 };  
22  
23 int main() {  
24     ImageCache cache;  
25  
26     {  
27         auto img1 = cache.get("photo.jpg"); // Charge depuis disque  
28         auto img2 = cache.get("photo.jpg"); // Trouve en cache  
29  
30     } // img1 et img2 detruits, Image liberee  
31  
32     auto img3 = cache.get("photo.jpg"); // Recharge depuis disque  
33     // Car l'image a ete liberee automatiquement !  
34 }
```

## 5 Tableau recapitulatif : Quel smart pointer utiliser ?

| Smart Pointer                 | Quand l'utiliser                            | Exemple                                                                  |
|-------------------------------|---------------------------------------------|--------------------------------------------------------------------------|
| <code>unique_ptr</code>       | Propriete exclusive, un seul proprietaire   | <code>unique_ptr&lt;Logger&gt; logger; //</code><br>Je possede ce logger |
| <code>shared_ptr</code>       | Propriete partagee, plusieurs proprietaires | <code>shared_ptr&lt;Data&gt; data; //</code><br>Plusieurs utilisateurs   |
| <code>weak_ptr</code>         | Observer sans posseder, briser les cycles   | <code>weak_ptr&lt;Parent&gt; parent; //</code><br>Observer le parent     |
| <code>T*</code> (raw pointer) | Pointer temporaire, pas de propriete        | <code>Widget* getWidget(); //</code> Je ne possede pas                   |

### Regle d'or

#### Par default : `unique_ptr`

Si vous avez besoin de copier : `shared_ptr`

Si vous avez une reference circulaire : `weak_ptr`

Si vous ne possédez PAS l'objet : pointeur brut (ou reference)

## 6 Exemples pratiques complets

### 6.1 Exemple 1 : Gestionnaire de plugins avec `unique_ptr`

```
1 class IPlugin {
2 public:
3     virtual std::string getName() const = 0;
4     virtual void execute() = 0;
5     virtual ~IPlugin() = default;
6 };
7
8 class PluginManager {
9 private:
10     std::map<std::string, std::unique_ptr<IPlugin>> plugins;
11
12 public:
13     // Prend la propriete du plugin
14     void addPlugin(std::unique_ptr<IPlugin> plugin) {
15         std::string name = plugin->getName();
16         plugins[name] = std::move(plugin);
17         std::cout << "Plugin " << name << " ajoute\n";
18     }
19
20     // Retourne un pointeur non-possesseur
21     IPlugin* getPlugin(const std::string& name) {
22         if (plugins.find(name) != plugins.end()) {
23             return plugins[name].get();
24         }
25         return nullptr;
26     }
27
28     void executeAll() {
29         for (auto& pair : plugins) {
30             pair.second->execute();
31         }
32     }
33
34     // Les plugins sont automatiquement detruits
35     // quand PluginManager est detruit
36 };
37
38 class HelloPlugin : public IPlugin {
39 public:
40     std::string getName() const override {
41         return "Hello";
42     }
43     void execute() override {
44         std::cout << "Hello from plugin!\n";
45     }
46 };
47
48 int main() {
49     PluginManager manager;
50
51     manager.addPlugin(std::make_unique<HelloPlugin>());
52
53     if (auto* plugin = manager.getPlugin("Hello")) {
54         plugin->execute();
55     }
```

```

55     }
56
57     manager.executeAll();
58
59     // Tous les plugins sont automatiquement detruits
60 }

```

## 6.2 Exemple 2 : Graphe avec shared\_ptr et weak\_ptr

```

1  class Node {
2  private:
3      std::string nom;
4      std::vector<std::shared_ptr<Node>> enfants;
5      std::weak_ptr<Node> parent; // weak pour eviter cycle !
6
7  public:
8      Node(const std::string& n) : nom(n) {
9          std::cout << "Noeud " << nom << " cree\n";
10     }
11
12     ~Node() {
13         std::cout << "Noeud " << nom << " detruit\n";
14     }
15
16     void ajouterEnfant(std::shared_ptr<Node> enfant) {
17         enfants.push_back(enfant);
18         enfant->parent = shared_from_this();
19     }
20
21     void afficher(int niveau = 0) const {
22         std::string indent(niveau * 2, ' ');
23         std::cout << indent << nom << "\n";
24         for (const auto& enfant : enfants) {
25             enfant->afficher(niveau + 1);
26         }
27     }
28
29     std::string getNom() const { return nom; }
30
31     void afficherParent() const {
32         if (auto p = parent.lock()) {
33             std::cout << nom << " a pour parent : "
34                 << p->getNom() << "\n";
35         } else {
36             std::cout << nom << " n'a pas de parent\n";
37         }
38     }
39 };
40
41 // IMPORTANT : Pour utiliser shared_from_this()
42 class Node : public std::enable_shared_from_this<Node> {
43     // ... meme code qu'au-dessus
44 };
45
46 int main() {
47     auto racine = std::make_shared<Node>("Racine");
48     auto enfant1 = std::make_shared<Node>("Enfant1");

```

```

49     auto enfant2 = std::make_shared<Node>("Enfant2");
50     auto petitEnfant = std::make_shared<Node>("PetitEnfant");
51
52     racine->ajouterEnfant(enfant1);
53     racine->ajouterEnfant(enfant2);
54     enfant1->ajouterEnfant(petitEnfant);
55
56     racine->afficher();
57     enfant1->afficherParent();
58     petitEnfant->afficherParent();
59
60     // Tous les noeuds sont automatiquement detruits
61     // dans le bon ordre grace a shared_ptr et weak_ptr
62 }

```

### 6.3 Exemple 3 : Factory retournant unique\_ptr

```

1  class Shape {
2  public:
3      virtual double area() const = 0;
4      virtual ~Shape() = default;
5  };
6
7  class Circle : public Shape {
8  private:
9      double rayon;
10 public:
11     Circle(double r) : rayon(r) {}
12     double area() const override {
13         return 3.14159 * rayon * rayon;
14     }
15 };
16
17 class Rectangle : public Shape {
18 private:
19     double largeur, hauteur;
20 public:
21     Rectangle(double l, double h) : largeur(l), hauteur(h) {}
22     double area() const override {
23         return largeur * hauteur;
24     }
25 };
26
27 class ShapeFactory {
28 public:
29     static std::unique_ptr<Shape> createCircle(double rayon) {
30         return std::make_unique<Circle>(rayon);
31     }
32
33     static std::unique_ptr<Shape> createRectangle(double l, double h) {
34         return std::make_unique<Rectangle>(l, h);
35     }
36
37     static std::unique_ptr<Shape> create(const std::string& type,
38                                         double param1,
39                                         double param2 = 0) {
40         if (type == "circle") {

```

```

41         return createCircle(param1);
42     } else if (type == "rectangle") {
43         return createRectangle(param1, param2);
44     }
45     return nullptr;
46 }
47 };
48
49 int main() {
50     std::vector<std::unique_ptr<Shape>> formes;
51
52     formes.push_back(ShapeFactory::create("circle", 5.0));
53     formes.push_back(ShapeFactory::create("rectangle", 4.0, 3.0));
54     formes.push_back(ShapeFactory::createCircle(3.0));
55
56     double aireTotatale = 0;
57     for (const auto& forme : formes) {
58         aireTotatale += forme->area();
59     }
60
61     std::cout << "Aire totale : " << aireTotale << "\n";
62
63     // Toutes les formes sont automatiquement detruites
64 }

```

## 7 Exercices pratiques

### 7.1 Exercice 1 : Convertir raw pointers en unique\_ptr (facile)

#### Enonce

Convertissez ce code utilisant des raw pointers en code utilisant unique\_ptr :

```
1 class Data {
2     int value;
3 public:
4     Data(int v) : value(v) {}
5     int getValue() const { return value; }
6 };
7
8 int main() {
9     Data* d1 = new Data(10);
10    Data* d2 = new Data(20);
11
12    std::cout << d1->getValue() << "\n";
13    std::cout << d2->getValue() << "\n";
14
15    delete d1;
16    delete d2;
17 }
```

#### Solution :

```
1 int main() {
2     auto d1 = std::make_unique<Data>(10);
3     auto d2 = std::make_unique<Data>(20);
4
5     std::cout << d1->getValue() << "\n";
6     std::cout << d2->getValue() << "\n";
7
8     // Pas de delete : automatique !
9 }
```

### 7.2 Exercice 2 : Gestionnaire de ressources (moyen)

#### Enonce

Creez une classe ResourceManager qui :

- Stocke des ressources (classe Resource) avec unique\_ptr
- Methode addResource(nom, unique\_ptr) pour ajouter
- Methode getResource(nom) qui retourne un pointeur brut
- Methode releaseResource(nom) qui retire et retourne le unique\_ptr

#### Solution :

```
1 class Resource {
2 private:
3     std::string nom;
4 public:
5     Resource(const std::string& n) : nom(n) {
6         std::cout << "Resource " << nom << " creee\n";
7     }
8 }
```

```

7     }
8     ~Resource() {
9         std::cout << "Resource " << nom << " detruite\n";
10    }
11    void use() {
12        std::cout << "Utilisation de " << nom << "\n";
13    }
14 };
15
16 class ResourceManager {
17 private:
18     std::map<std::string, std::unique_ptr<Resource>> resources;
19
20 public:
21     void addResource(const std::string& nom,
22                     std::unique_ptr<Resource> res) {
23         resources[nom] = std::move(res);
24     }
25
26     Resource* getResource(const std::string& nom) {
27         if (resources.find(nom) != resources.end()) {
28             return resources[nom].get();
29         }
30         return nullptr;
31     }
32
33     std::unique_ptr<Resource> releaseResource(const std::string& nom) {
34         auto it = resources.find(nom);
35         if (it != resources.end()) {
36             auto res = std::move(it->second);
37             resources.erase(it);
38             return res;
39         }
40         return nullptr;
41     }
42 };
43
44 int main() {
45     ResourceManager manager;
46
47     manager.addResource("texture",
48                         std::make_unique<Resource>("texture.png"));
49     manager.addResource("model",
50                         std::make_unique<Resource>("model.obj"));
51
52     if (auto* res = manager.getResource("texture")) {
53         res->use();
54     }
55
56     auto released = manager.releaseResource("model");
57     // released possede maintenant la ressource
58
59     // texture detruite automatiquement
60     // model detruit quand released sort de la portee
61 }

```

### 7.3 Exercice 3 : Arbre avec shared\_ptr (difficile)

#### Enonce

Creez une structure d'arbre binaire ou :

- Chaque noeud a deux enfants (gauche, droite) avec shared\_ptr
- Chaque noeud a un parent avec weak\_ptr
- Methode insert(valeur) pour inserer
- Methode contains(valeur) pour chercher
- Methode afficherParent() pour afficher le parent d'un noeud

#### Solution partielle :

```
1 class TreeNode : public std::enable_shared_from_this<TreeNode> {
2 private:
3     int valeur;
4     std::shared_ptr<TreeNode> gauche;
5     std::shared_ptr<TreeNode> droite;
6     std::weak_ptr<TreeNode> parent;
7
8 public:
9     TreeNode(int v) : valeur(v) {}
10
11     void insert(int v) {
12         if (v < valeur) {
13             if (!gauche) {
14                 gauche = std::make_shared<TreeNode>(v);
15                 gauche->parent = shared_from_this();
16             } else {
17                 gauche->insert(v);
18             }
19         } else {
20             if (!droite) {
21                 droite = std::make_shared<TreeNode>(v);
22                 droite->parent = shared_from_this();
23             } else {
24                 droite->insert(v);
25             }
26         }
27     }
28
29     bool contains(int v) const {
30         if (v == valeur) return true;
31         if (v < valeur && gauche) return gauche->contains(v);
32         if (v > valeur && droite) return droite->contains(v);
33         return false;
34     }
35
36     void afficherParent() const {
37         if (auto p = parent.lock()) {
38             std::cout << valeur << " a pour parent : "
39                 << p->valeur << "\n";
40         } else {
41             std::cout << valeur << " est la racine\n";
42         }
43     }
44 }
```

```

44 };
45
46 int main() {
47     auto racine = std::make_shared<TreeNode>(10);
48     racine->insert(5);
49     racine->insert(15);
50     racine->insert(3);
51     racine->insert(7);
52
53     std::cout << "Contient 7 ? "
54               << (racine->contains(7) ? "oui" : "non") << "\n";
55     std::cout << "Contient 12 ? "
56               << (racine->contains(12) ? "oui" : "non") << "\n";
57 }

```

## 8 Pièges et bonnes pratiques

### 8.1 Piège 1 : Créer deux `shared_ptr` indépendants

#### MAUVAIS

```
1 int* raw = new int(42);
2
3 std::shared_ptr<int> ptr1(raw);
4 std::shared_ptr<int> ptr2(raw); // ERREUR ! Deux control blocks !
5
6 // Quand ptr1 est détruit, il delete raw
7 // Quand ptr2 est détruit, il delete raw ENCORE
8 // -> DOUBLE DELETE -> CRASH
```

#### BON

```
1 auto ptr1 = std::make_shared<int>(42);
2 auto ptr2 = ptr1; // Copie : partage le meme control block
3 // Un seul control block, compteur = 2
```

### 8.2 Piège 2 : Retourner `this` dans `shared_ptr`

#### MAUVAIS

```
1 class Widget {
2 public:
3     std::shared_ptr<Widget> getThis() {
4         return std::shared_ptr<Widget>(this); // ERREUR !
5         // Cree un nouveau control block independant
6     }
7 };
```

```
1 class Widget : public std::enable_shared_from_this<Widget> {
2 public:
3     std::shared_ptr<Widget> getThis() {
4         return shared_from_this(); // OK !
5         // Utilise le control block existant
6     }
7 };
```

(

### 8.3 Piège 3 : Stocker `shared_ptr` dans un `weak_ptr` trop tard

```
1 int value = 42 // std::shared_ptr<int> shared(value) // DANGEREUX // weak =
   // shared // value détruit // weak point vers mémoire invalide
2 // SOLUTION Toujours créer avec
   make_shared / auto_shared = std::make_shared<int>(42) / weak = shared
3 ;:::
```

## 8.4 Bonne pratique 1 : Preferer `make_unique` et `make_shared`

```
1 // EVITER
   stdunique_ptr< Widget > ptr(newWidget())stdshared_ptr< Widget > sptr(newWidget())
2 :::
```

### Raisons :

- Plus court et lisible
- Exception-safe
- Pour `shared_ptr` : une seule allocation au lieu de deux

## 8.5 Bonne pratique 2 : Passer les smart pointers

```
3 // Partager la propriete void sharesOwnership(stdshared_ptr< Widget >
   widget)//AppelsharesOwnership(myWidget)//Copie
4 // Utiliser sans posseder void usesWidget(const Widget widget) void
   usesWidget(Widget* widget) // Appel usesWidget(*mySmartPointer) ou
   usesWidget(mySmartPointer.get())
5 :::::;;;
```

## 9 Quiz de validation

### 9.1 Questions

1. Quelle est la difference entre `unique_ptr` et `shared_ptr` ?
2. Peut-on copier un `unique_ptr` ? Et un `shared_ptr` ?
3. Comment transferer la propriete d'un `unique_ptr` ?
4. A quoi sert `weak_ptr` ?
5. Que fait `shared_ptr.use_count()` ?
6. Pourquoi utiliser `make_unique` plutot que `new` ?
7. Qu'est-ce qu'une reference circulaire et comment la resoudre ?
8. Comment obtenir un pointeur brut depuis un `unique_ptr` ?
9. Que fait `weak_ptr.lock()` ?
10. Quel est le cout en memoire d'un `shared_ptr` par rapport a un raw pointer ?

### 9.2 Reponses

1. `unique_ptr` : propriete exclusive (un seul proprietaire). `shared_ptr` : propriete partagee (plusieurs proprietaires).
2. `unique_ptr` : non, on ne peut pas le copier (seulement le move). `shared_ptr` : oui, on peut le copier.
3. Avec `std::move(ptr)`. Apres le move, l'ancien proprietaire devient vide.
4. `weak_ptr` sert a observer un objet sans le posseder. Utile pour briser les references circulaires.
5. `use_count()` retourne le nombre de `shared_ptr` qui possedent l'objet.
6. `make_unique` est plus sur (exception-safe), plus court, et plus lisible.
7. Reference circulaire : deux `shared_ptr` qui se pointent mutuellement. Solution : remplacer l'un par un `weak_ptr`.
8. Avec la methode `get()` : `Widget* raw = uniquePtr.get();`
9. `lock()` essaie de creer un `shared_ptr` temporaire. Retourne un `shared_ptr` valide si l'objet existe, sinon `nullptr`.
10. `shared_ptr` utilise deux pointeurs (objet + control block) au lieu d'un. Donc environ le double de la taille.

## 10 Pour aller plus loin

### 10.1 Custom deleters

Vous pouvez spécifier un deleter personnalisé :

```
1 FILE* f)
  if (f)
    fclose(f);
    std::cout << "Fichier ferme";
    ;
    std::unique_ptr<FILE, decltype(fileDeleter)> file(
    fopen("data.txt", "r"),
    fileDeleter
    );
    // Le fichier sera ferme automatiquement avec fclose
```

### 10.2 enable\_shared\_from\_this

Pour obtenir un shared\_ptr vers this :

```
1 class Widget : public std::enable_shared_from_this<Widget> {
2 public:
3     std::shared_ptr<Widget> getSelf() {
4         return shared_from_this();
5     }
6 };
```

### 10.3 Atomic shared\_ptr (C++20)

Pour utiliser shared\_ptr de façon thread-safe :

```
1 #include <atomic>
2
3 std::atomic<std::shared_ptr<int>>> atomicPtr;
4 // Operations atomiques sur shared_ptr
```

## Fin de la Seance 4

Prochaine seance : Templates et Programmation Generique